```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.UI;
using Random = UnityEngine.Random;
using TMPro;

// Set of game states to determine game flow
public enum GameState { dealerTurn, playerTurn, endGame }

public class GameController : MonoBehaviour
{
    // References game states
    public GameState gameState;

    // Button inputs to interact with game
    [Header("UI")]
        public GameObject hitButton;
        public GameObject standButton;
        public GameObject restartButton;
        public GameObject quitButton;
        public TextMeshProUGUI endGameText;
    [Space(10)]

    [Header("Cards")]
        // Holds card objects
        public GameObject dealerHandObject;
        public GameObject playerHandObject;
        // Lists to hold deck and hands
        public List<Card> deck = new List<Card>();
        public List<Card> dealerHand = new List<Card>();
        public List<Card> playerHand = new List<Card>();

    // Holds values of hands
    public int dealerHandValue;
    public int playerHandValue;

    // Card display offsets
    float playerCardOffsetX = 0.0f;
    float playerCardOffsetY = 0.0f;
    float dealerCardOffsetX = 0.0f;
    float dealerCardOffsetY = 0.0f;

    public void Start()
    {
        // Ensures every card can spawn on game start
        foreach (Card card in deck)
            card.isSpawned = false;

        // Draws initial cards
        DrawPlayer(2);
        DrawDealer(2);
        // Adds drawn cards to screen
        DeckVisualUpdater(playerHand, playerHandObject);
        DeckVisualUpdater(dealerHand, dealerHandObject);
        // Clears end text and sets game to player turn
        endGameText.text = "";
        gameState = GameState.playerTurn;
    }

    public void Update()
    {
        // Checks ace value
        CheckAce();
```

```
        // Determines if game is over
        if (playerHandValue >= 21 || dealerHandValue > 21)
        {
            DeckVisualUpdater(playerHand, playerHandObject);
            DeckVisualUpdater(dealerHand, dealerHandObject);
            gameState = GameState.endGame;
        }

        // Logic for if it is the player turn
        if (gameState == GameState.playerTurn)
        {
            hitButton.SetActive(true);
            standButton.SetActive(true);
        }
        // Logic for if it is the dealer turn
        else if (gameState == GameState.dealerTurn)
        {
            hitButton.SetActive(false);
            standButton.SetActive(false);
            StartCoroutine(DealerTurn());
        }
        // Logic for if the game is over
        else
            EndGame();
    }

    public void DrawPlayer(int amount)
    {
        // Draws an amount of cards from the deck then adds them to the
respective hand
        for (int i = 0; i < amount; i++)
        {
            int randomCardNumber = Random.Range(0, deck.Count);
            Card randomCard = deck[randomCardNumber];
            deck.RemoveAt(randomCardNumber);
            playerHand.Add(randomCard);
            // Determines if card is an ace
            if (randomCard.value == 1)
                playerHandValue += randomCard.aceValue;
            else
                playerHandValue += randomCard.value;
        }
    }

    public void DrawDealer(int amount)
    {
        // Draws an amount of cards from the deck then adds them to the
respective hand
        for (int i = 0; i < amount; i++)
        {
            int randomCardNumber = Random.Range(0, deck.Count);
            Card randomCard = deck[randomCardNumber];
            deck.RemoveAt(randomCardNumber);
            dealerHand.Add(randomCard);
            // Determines if card is an ace
            if (randomCard.value == 1)
                dealerHandValue += randomCard.aceValue;
            else
                dealerHandValue += randomCard.value;
        }
    }

    public void PlayerTurnHit()
```

```
    {
        // Deals the player a card
        DrawPlayer(1);
        // Updates game visually and logically
        DeckVisualUpdater(playerHand, playerHandObject);
        gameState = GameState.dealerTurn;
    }

    public void PlayerTurnStand()
    {
        // Passes the players turn and updates game visually
        DeckVisualUpdater(playerHand, playerHandObject);

        // Ends game if dealer cannot draw anymore
        if (dealerHandValue >= 16)
            gameState = GameState.endGame;
        else
            gameState = GameState.dealerTurn;
    }

    public IEnumerator DealerTurn()
    {
        yield return new WaitForSeconds(1f);
        // Dealer draws if below 16, else passes turn
        if (dealerHandValue < 16)
            DrawDealer(1);

        // Updates the game visually and logically
        DeckVisualUpdater(dealerHand, dealerHandObject);
        gameState = GameState.playerTurn;
    }

    public void EndGame()
    {
        // Ends game, and displays end screens
        hitButton.SetActive(false);
        standButton.SetActive(false);
        restartButton.SetActive(true);
        quitButton.SetActive(true);
        DeckVisualUpdater(playerHand, playerHandObject);
        DeckVisualUpdater(dealerHand, dealerHandObject);

        // Logic to display correct end text
        if (playerHandValue > 21)
            endGameText.text = "You Lose";
        else if (dealerHandValue > 21)
            endGameText.text = "You Win";
        else
        {
            if (playerHandValue > dealerHandValue)
                endGameText.text = "You Win";
            else if (dealerHandValue > playerHandValue)
                endGameText.text = "You lose";
            else
                endGameText.text = "Tie";
        }
    }

    public void CheckAce()
    {
        // Goes through each card in hand and if ace, and is able, then ace
value is changed
        foreach (Card card in playerHand)
        {
```

```
            if (card.value == 1 && playerHandValue > 21 && !card.aceValueCheck)
            {
                // Checker so ace cannot infinitely decrease deck value
                card.aceValueCheck = true;
                playerHandValue -= card.aceValue;
                playerHandValue += card.value;
            }
        }

        foreach (Card card in dealerHand)
        {
            if (card.value == 1 && dealerHandValue > 21 && !card.aceValueCheck)
            {
                card.aceValueCheck = true;
                dealerHandValue -= card.aceValue;
                dealerHandValue += card.value;
            }
        }
    }

    public void DeckVisualUpdater(List<Card> hand, GameObject handHolder)
    {
        // Goes through each card in hand and spawns in its game object if not
already spawned
        foreach (Card card in hand)
        {
            if (!card.isSpawned)
            {
                GameObject spawnedCard = Instantiate(card.cardObject);
                spawnedCard.transform.SetParent(handHolder.transform);
                if (handHolder.tag == "Player")
                {
                    spawnedCard.transform.localPosition = new
Vector3(playerCardOffsetX, playerCardOffsetY, 0);
                    playerCardOffsetX += 0.5f;
                    playerCardOffsetY += 0.1f;
                }
                else
                {
                    spawnedCard.transform.localPosition = new
Vector3(dealerCardOffsetX, dealerCardOffsetY, 0);
                    dealerCardOffsetX += 0.5f;
                    dealerCardOffsetY += 0.1f;
                }

                card.isSpawned = true;

                // Rotates dealer cards
                if (handHolder.tag != "Player")
                    if (spawnedCard.transform.position.x != 0)
                        spawnedCard.transform.rotation = new Quaternion(0, 0,
180, 0);
            }
        }

        // Unflips dealer cards
        if (gameState == GameState.endGame)
        {
            for (int i = 0; i < handHolder.transform.childCount; i++)
            {
                if (handHolder.transform.GetChild(i).transform.rotation.z != 0)
                    handHolder.transform.GetChild(i).transform.rotation = new
Quaternion(0, 0, 0, 0);
            }
```

```
            }
        }
    }
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;

[CreateAssetMenu(fileName = "Card", menuName = "ScriptableObjects/Card")]
public class Card : ScriptableObject
{
    // Card asset by Game Asset Studio:
https://assetstore.unity.com/packages/3d/props/tools/free-playing-cards-pack-
154780
    [Header("Card Info")]
        public int value;               // Values of card
        public int aceValue;            // Ace value
        public bool aceValueCheck;      // Ace checker to prevent bug
        public bool isSpawned;          // Makes sure card cant be spawned on
screen more than once
        public GameObject cardObject;   // Card game object
}
```

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.SceneManagement;

public class SceneController : MonoBehaviour
{
    // Changes to game scene
    public void ToGame()
    {
        SceneManager.LoadScene("Game");
    }

    // Changes to title scene
    public void ToTitle()
    {
        SceneManager.LoadScene("Title");
    }

    // Restarts current scene
    public void RestartScene()
    {
        SceneManager.LoadScene(SceneManager.GetActiveScene().buildIndex);
    }
}
```